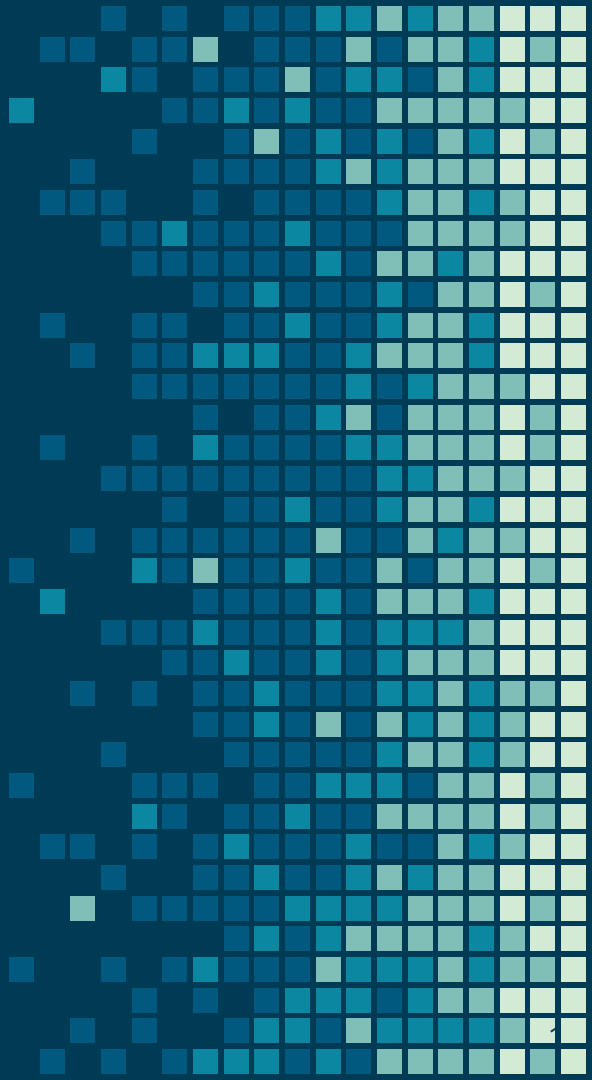
EMBEDDED CONTROL

Prof. Luca De Cicco
luca.decicco@poliba.it
https://c3lab.poliba.it/LDC

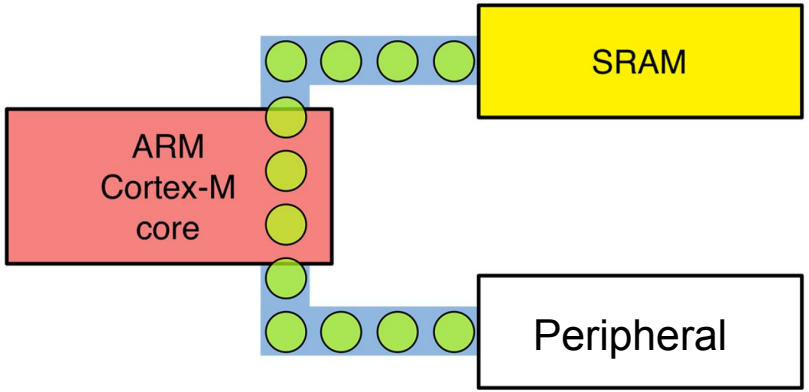


DMA data transfer and Analog to Digital Conversion

DMA basics

Why Direct Memory Access is useful

- We have explored access in polling mode an interrupt mode
- Interrupt better than polling CPU-wise, however data exchange between peripheral and SRAM is always under the control of the CPU Core
- When data rate is high it is better that peripheral and SRAM communicate directly



STM32 DMA

In every STM32 MCU, the DMA controller:

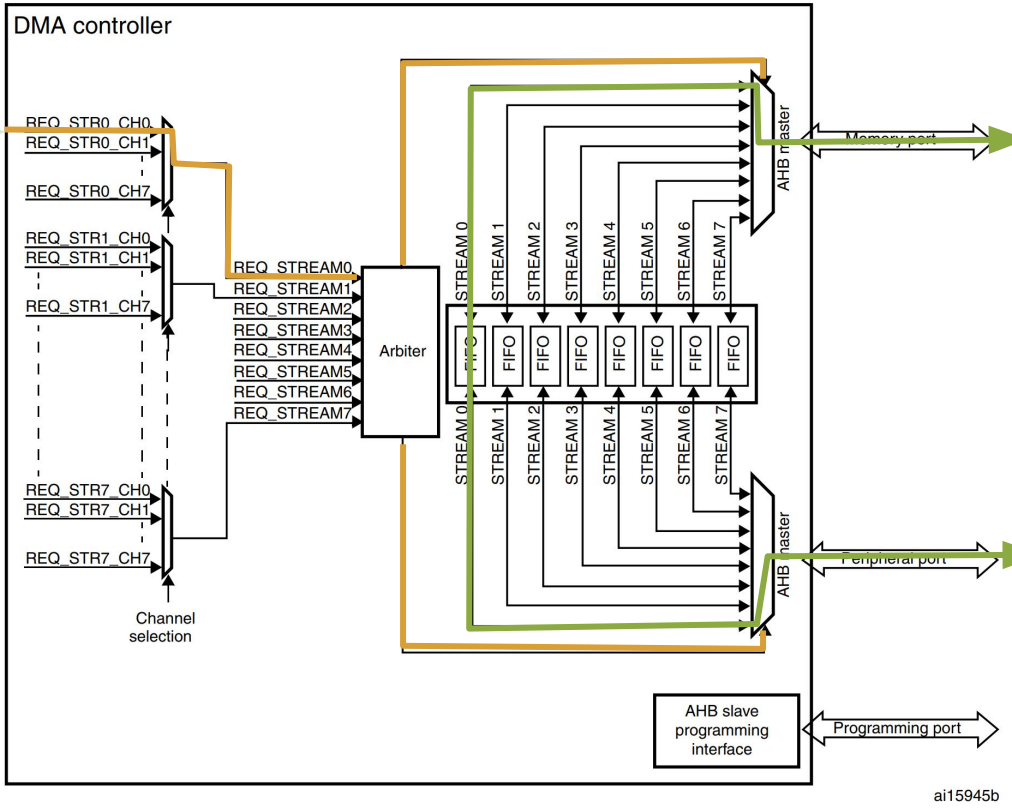
- has two master ports connected to the **AHB** bus:
  - peripheral: to access a slave peripheral
  - memory: to access a memory controller (can be external)
- one slave port used to program the DMA from its master, i.e., the CPU
- a number of **DMA channels** each connectable to a specific "peripheral request line" (more on this later)
- allows data flowing in both directions: memory to peripheral, peripheral to memory

STM32F2/F4/F7 DMA

- Different MCUs have different DMA implementations
- STM32F446RE has two DMAs each with 8 **streams**
- Each **stream** is dedicated to managing memory access requests from one or more peripherals
- Each stream can have up to 8 **channels** (requests) in total, but only one channel can be active in a stream
- An **arbiter** assigns the DMA to a specific channel at a given time to allow data exchange between a specific peripheral and the memory

Simplified DMA structure

A request from channel 0 of stream 0 is issued



Data between Memory and selected peripheral occurs

How requests are mapped to peripherals?

Answer (always!) in reference manual (pg. 207)

Data reception from **UART5** in DMA mode occurs on **Channel 4** of **Stream 0**

Table 28. DMA1 request mapping

Peripheral requests	Stream 0	Stream 1	Stream 2	Stream 3	Stream 4	Stream 5	Stream 6	Stream 7
Channel 0	SPI3_RX	SPDIFRX_DT	SPI3_RX	SPI2_RX	SPI2_TX	SPI3_TX	SPDIFRX_CS	SPI3_TX
Channel 1	I2C1_RX	I2C3_RX	TIM7_UP	-	TIM7_UP	I2C1_RX	I2C1_TX	I2C1_TX
Channel 2	TIM4_CH1	-	FMPI2C1_RX	TIM4_CH2	-	FMPI2C1_RX	TIM4_UP	TIM4_CH3
Channel 3	-	TIM2_UP TIM2_CH3	I2C3_RX	-	I2C3_TX	TIM2_CH1	TIM2_CH2 TIM2_CH4	TIM2_UP TIM2_CH4
Channel 4	UART5_RX	USART3_RX	UART4_RX	USART3_TX	UART4_TX	USART2_RX	USART2_TX	UART5_TX
Channel 5	-	-	TIM3_CH4 TIM3_UP	-	TIM3_CH1 TIM3_TRIG	TIM3_CH2	-	TIM3_CH3
Channel 6	TIM5_CH3 TIM5_UP	TIM5_CH4 TIM5_TRIG	TIM5_CH1	TIM5_CH4 TIM5_TRIG	TIM5_CH2	-	TIM5_UP	-
Channel 7	-	TIM6_UP	I2C2_RX	I2C2_RX	USART3_TX	DAC1	DAC2	I2C2_TX

DMA 2

Table 29. DMA2 request mapping

Peripheral requests	Stream 0	Stream 1	Stream 2	Stream 3	Stream 4	Stream 5	Stream 6	Stream 7
Channel 0	ADC1	SAI1_A	TIM8_CH1 TIM8_CH2 TIM8_CH3	SAI1_A	ADC1	SAI1_B	TIM1_CH1 TIM1_CH2 TIM1_CH3	SAI2_B
Channel 1	-	DCMI	ADC2	ADC2	SAI1_B	-	-	DCMI
Channel 2	ADC3	ADC3	-	-	-	-	-	-
Channel 3	SPI1_RX	-	SPI1_RX	SPI1_TX	SAI2_A	SPI1_TX	SAI2_B	QUADSPI
Channel 4	SPI4_RX	SPI4_TX	USART1_RX	SDIO		USART1_RX	SDIO	USART1_TX
Channel 5	-	USART6_RX	USART6_RX	SPI4_RX	SPI4_TX	-	USART6_TX	USART6_TX
Channel 6	TIM1_TRIG	TIM1_CH1	TIM1_CH2	TIM1_CH1	TIM1_CH4 TIM1_TRIG TIM1_COM	TIM1_UP	TIM1_CH3	-
Channel 7	-	TIM8_UP	TIM8_CH1	TIM8_CH2	TIM8_CH3	-	-	TIM8_CH4 TIM8_TRIG TIM8_COM

Programming DMA in HAL

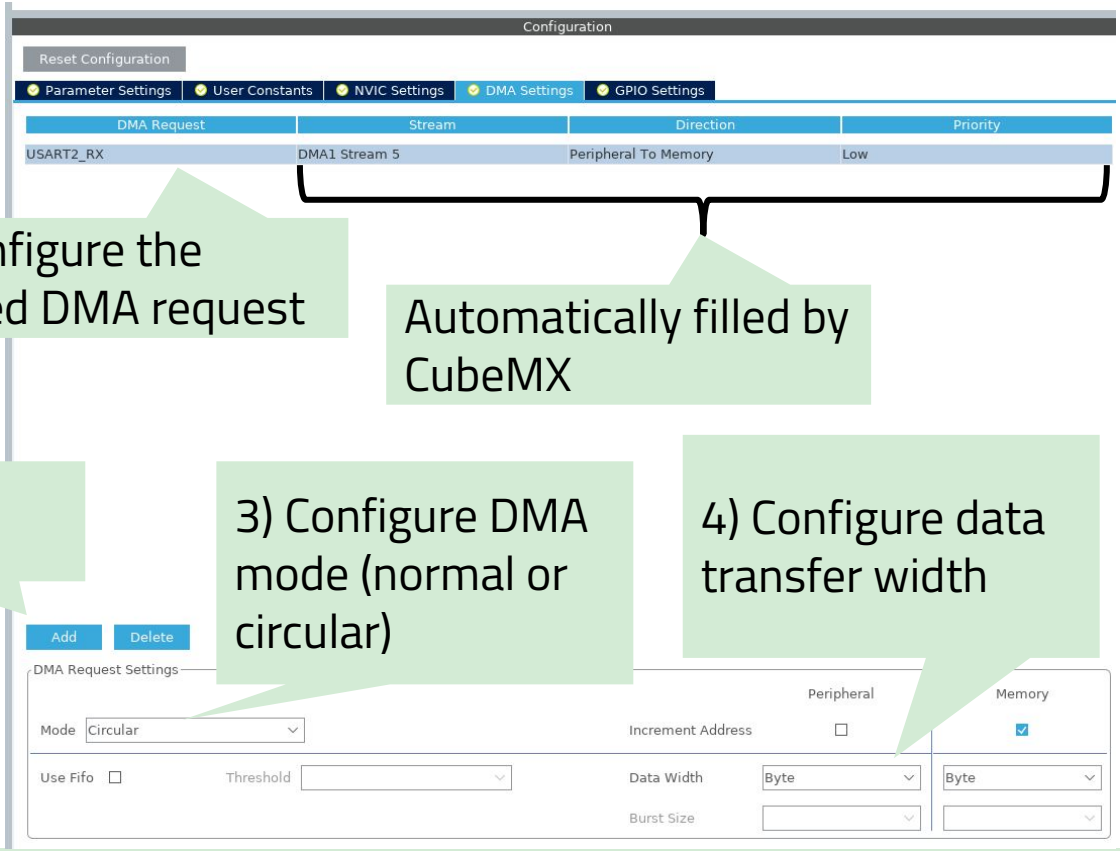
- All the HAL functions related to DMA accept as first parameter an instance of the C struct `DMA_HandleTypeDef`
- Configuring DMA programmatically can be quite daunting, fortunately CubeMX can help us
- DMA Modes:
  - **Normal**: one request is performed, when transfer finishes DMA must be "rearmed" to request a new data transfer
  - **Circular**: requests are performed continuously without the need to programmatically "rearm" the data transfer

DMA configuration using CubeMX

Configuration is very easy:

- Go to the peripheral (example USART2) that you want to access in DMA mode
- In the “configuration” panel go to “DMA Settings”
- Click on “ADD” and configure the added item by selecting the desired “DMA request” (f.i., USART2_RX)
- In the “DMA Request Settings” configure the DMA mode (f.i., circular) and the data width of a transfer (can be byte, half word, or word)

Receiving data in DMA mode on USART2



Generated DMA configuration code

In usart.c look at `HAL_UART_MspInit` function:

```
hdma_usart2_rx.Instance = DMA1_Stream5;
hdma_usart2_rx.Init.Channel = DMA_CHANNEL_4;
hdma_usart2_rx.Init.Direction = DMA_PERIPH_TO_MEMORY;
hdma_usart2_rx.Init.PeriphInc = DMA_PINC_DISABLE;
hdma_usart2_rx.Init.MemInc = DMA_MINC_ENABLE;
...
hdma_usart2_rx.Init.Mode = DMA_CIRCULAR;
hdma_usart2_rx.Init.Priority = DMA_PRIORITY_LOW;
hdma_usart2_rx.Init.FIFOMode = DMA_FIFOMODE_DISABLE;
HAL_DMA_Init(&hdma_usart2_rx); /* Initializes DMA */
__HAL_LINKDMA(uartHandle,hdmarx,hdma_usart2_rx); /* Links to
                                                    peripheral */
```

DMA peripheral access with HAL

- Once the peripheral has been configured to use DMA, peripheral to memory (or vice-versa) transmission can be started with a function typically of this form:

```
HAL_PERIPH_TRANSMTYPE_DMA(handler, dataptr, size)
```

Example: sending 128 bytes in DMA mode on UART2

```
uint8_t data[128];
HAL_UART_Transmit_DMA(&huart2, data, sizeof(data));
```

- **Note**: if UART DMA is configured in **circular** mode, `data` will be continuously sent with the **currently stored** values

Analog to Digital Conversion

Acquiring analog signals

- Analog peripherals are still quite common
- Examples: analog sensors, potentiometers, transducers and audio peripherals
- These peripherals produce voltages in a given range
- To interface (read) these devices to MCU a peripheral must be used to convert an analog voltage level to a digital representation
- Analog-to-Digital Converters are electronic circuits that serve to this purpose

ADC circuits

- Several approaches to implement an ADC
- Most common approaches: *Delta Sigma, Successive Approximation Register*, *pipelined ADC*
- ADC can be characterized by several parameters:
  - resolution (in bits): the amount of bits used to represent the analog input (12-bit - 24-bit)
  - bandwidth measured in mega samples per second
  - power dissipation: the lower the better for applications running on batteries (f.i., drones, or mobile robots)

Choosing an ADC

- As always depends on the application and specifics in terms of budget (ADCs can be expensive)
- For specific applications using ADC provided by the MCU could be insufficient in terms of performances
- Sigma-Delta are used in applications requiring low sampling rates (~100 kSPS) and 12-bit to 24-bit resolution
- SAR ADCs are suitable for low power apps requiring moderate sampling rates (max ~5 MSPS)
- Pipelined are used for high-speed apps (digital oscilloscope, radar communication) with high sampling rates (> 5MSPS) with lower resolutions (~18-bits)

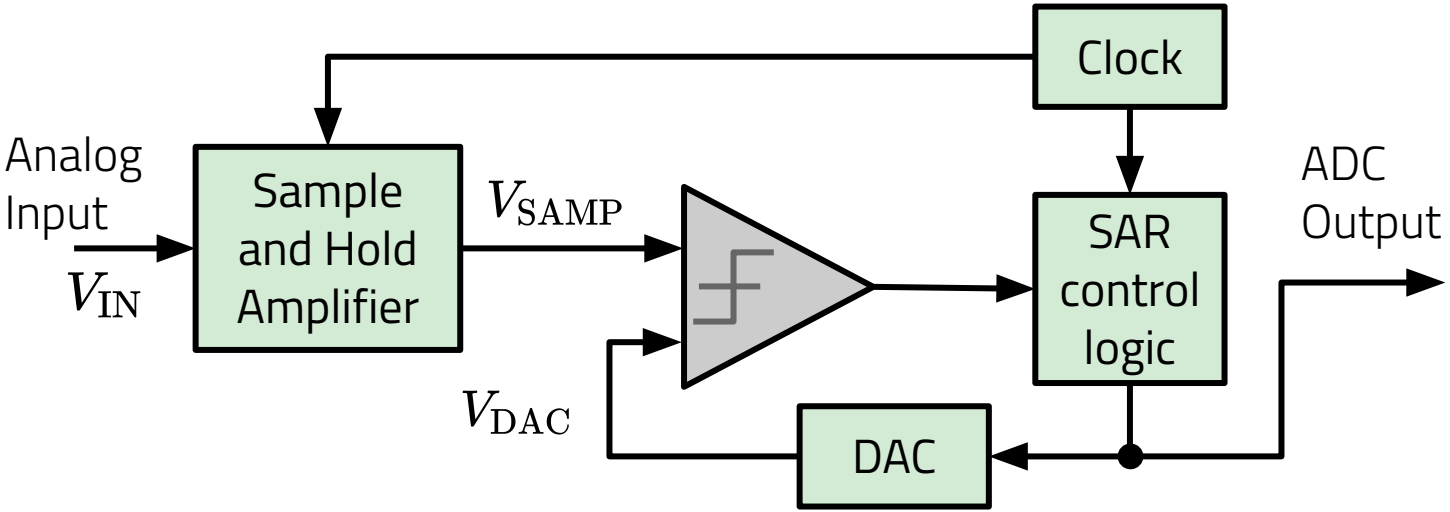
Understanding how quantization works

- The analog input (a “continuous” voltage) must be mapped to a digital representation with a given number of bits
- The analog input is mapped to an integer from 0 to 2^N-1
- The output the ADC is:

$$\mathrm{ADC_OUT} = \frac{V_{\mathrm{IN}}}{V_{\mathrm{REF}}} \cdot 2^N$$

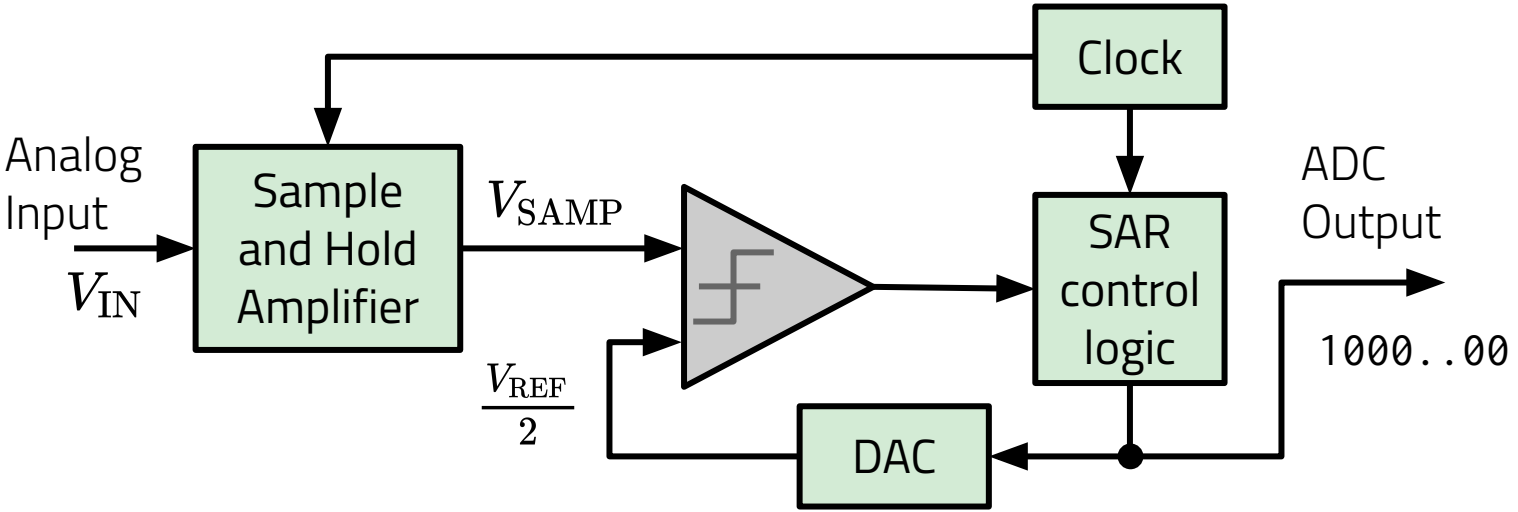
- V_{REF} is the reference voltage that is used to scale the input voltage ($0<V_{IN}<V_{REF}$), typically equal to V_{DD} (3.3V)
- The resolution in terms of voltage is thus $V_{REF}/2^N$
- For instance V_{REF}=3.3V and N=12 we have **V_{RES}=0.805mV**
- This is the maximum quantization error

Digital Quantization with SAR ADCs



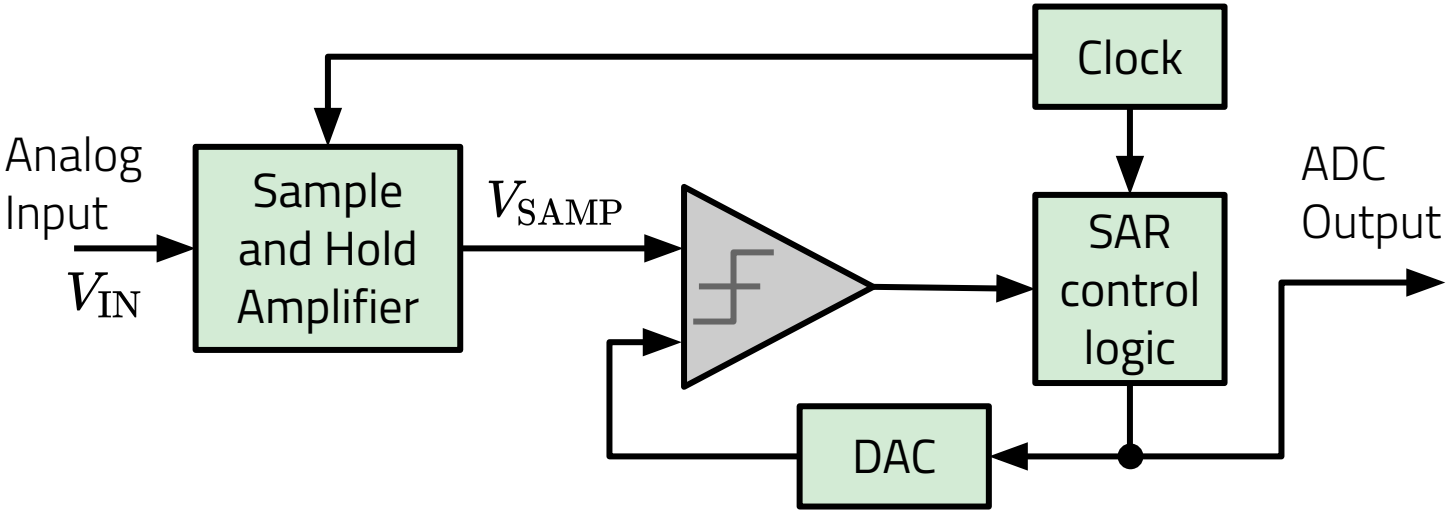
- A “binary search” algorithm is used to find the digital number (the ADC output) that better represents the analog input

The binary search algorithm (step 1)



- The algo starts by setting to 1 the MSB of ADC output
- We are representing 2^{N-1}, the output of DAC is $V_{DAC} = \frac{V_{REF}}{2^N} \cdot 2^{N-1} = \frac{V_{REF}}{2}$
- If the comparator senses $V_{IN} > \frac{V_{REF}}{2}$ the SAR logic does not change the MSB, otherwise it set the MSB to 0

The binary search algorithm (step 2)

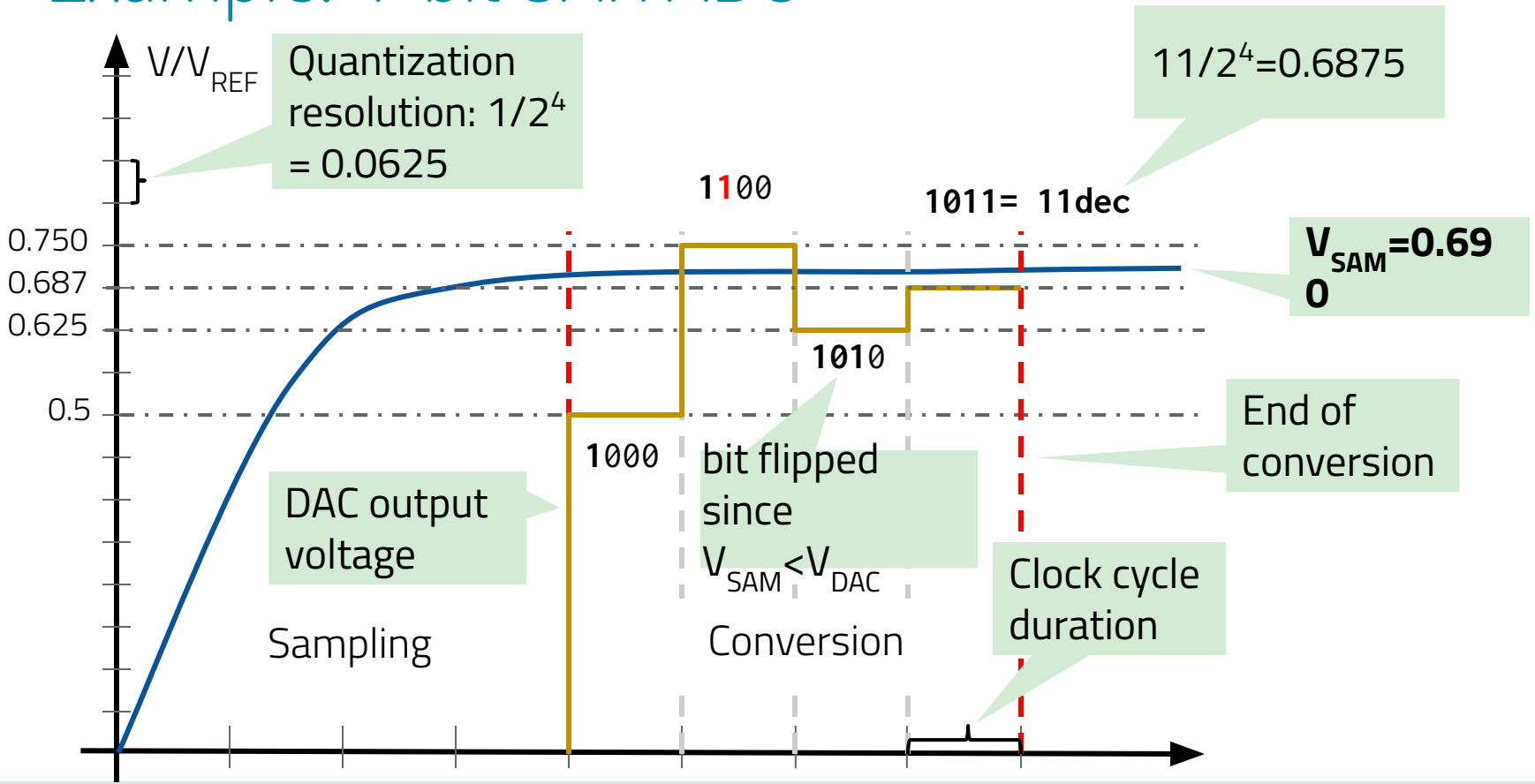


- Now the algo sets the (N-1)-th bit of ADC output to 1
- If the comparator senses $V_{IN} > V_{DAC}$ the SAR logic does not change the (N-1)-th bit, otherwise it set it to 0
- The algorithm continues until all the N bits are filled

ADC latency = sampling time + conversion time

- **Sampling time**:
  - depends on the SHA circuit capacitor value C_{ADC} and the value of the SHA total resistance
  - Sampling time must be larger than the settling time that is a multiple of the RC time constant
  - In practice a fixed amount of clock cycles is used
  - In STM32 ADCs sampling time is 3-4 clock cycles
- **Conversion time**: it takes N clock cycles
- Trade-off: better accuracy = larger latency!
- Conversion time: the higher the ADC clock frequency, the lower the conversion time

Example: 4-bit SAR ADC



STM32 ADCs

- ADC peripherals provided by STM32 family have a max resolution of 12-bit (STM32F3 has some 16-bit ADCs)
- The oversampling technique can be used to increase the resolution to 16-bit
- At least one ADC per MCU (STM32F446RE has 3 ADC)
- Each ADC has **multiple channels** which can be used to convert different analog voltages
- Each active channel of the same ADC peripheral is scanned in a particular order based on the "**rank**"
- Some value lines (F3 and L4) support differential ADC inputs (+ and -)

ADC HAL module

As any other peripheral, to access ADC using HAL library an handle is needed:

```
typedef struct {
    ADC_TypeDef *Instance;
    ADC_InitTypeDef Init;
    __IO uint32_t NbrOfCurrentConversionRank;
    DMA_HandleTypeDef *DMA_Handle;
    HAL_LockTypeDef Lock;
    __IO uint32_t State;
    __IO uint32_t ErrorCode;
} ADC_HandleTypeDef;
```

Pointer to peripheral registers

ADC configuration parameters

Similar to other HAL peripherals

`ADC_InitTypeDef Init` relevant fields

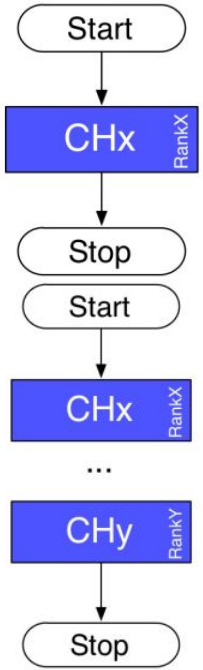
- `ClockPrescaler`: prescaler used to divide the clock feeding the ADC (`ADCCLK`). The higher the prescaler the lower the sampling frequency and the number of samples per second (value: 2 to 8)
- `Resolution`: sets the resolution in bits of ADC (`ADC_RESOLUTION_XB` where `X = 6,8,10,12`)
- Other fields are related to the "Conversion mode" employed (next slide)

Conversion modes: single conversion

STM32 MCUs implement several conversion modes useful to deal with different application scenarios:

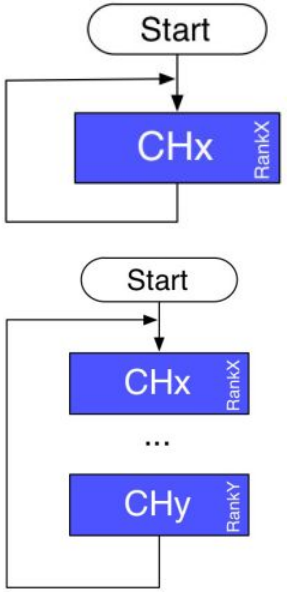
- **Single-Channel, Single Conversion Mode**: the ADC performs the single conversion (single sample) of a single channel

- **Scan Single Conversion Mode** (multichannel) read any sequence of up to 16 channels successively according to defined “rank”



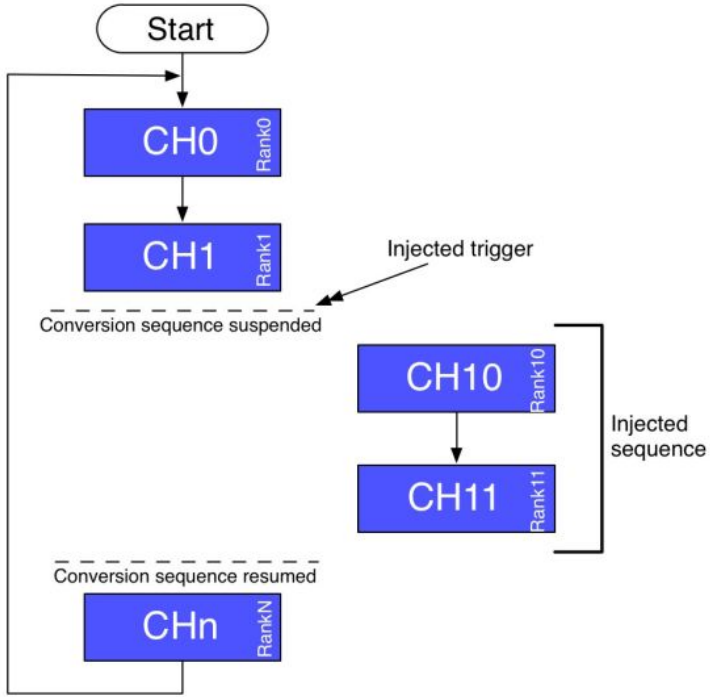
Conversion modes: continuous conversion

- **Single-Channel, Continuous Conversion Mode**: a given channel is continuously read in background without any intervention from the CPU
- **Scan Continuous Conversion Mode**: configured channels are continuously read in the order specified by ranks



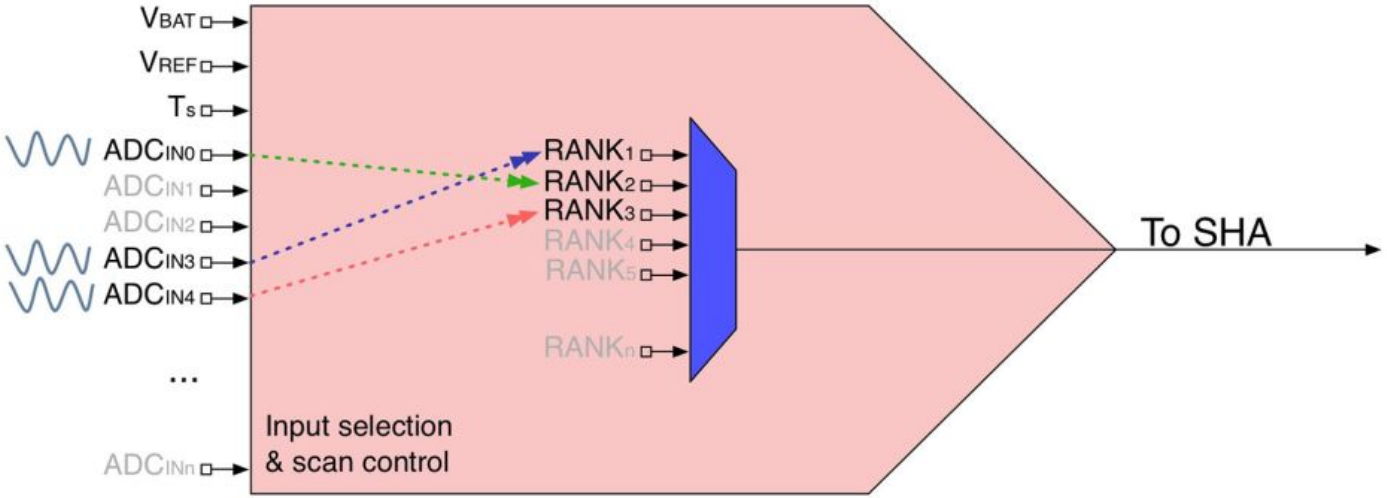
Conversion Mode: injected conversion

- Used when conversion is **triggered** by an external event or by software
- The **injected group has priority** over the regular channel group
- It interrupts the conversion of the current channel in the regular channel group
- External event might be, f.i., a timer



Channel ranks

Channel ranks allow to “order” channel scans to perform a given sequence when ADC is configured in a “Scan” mode



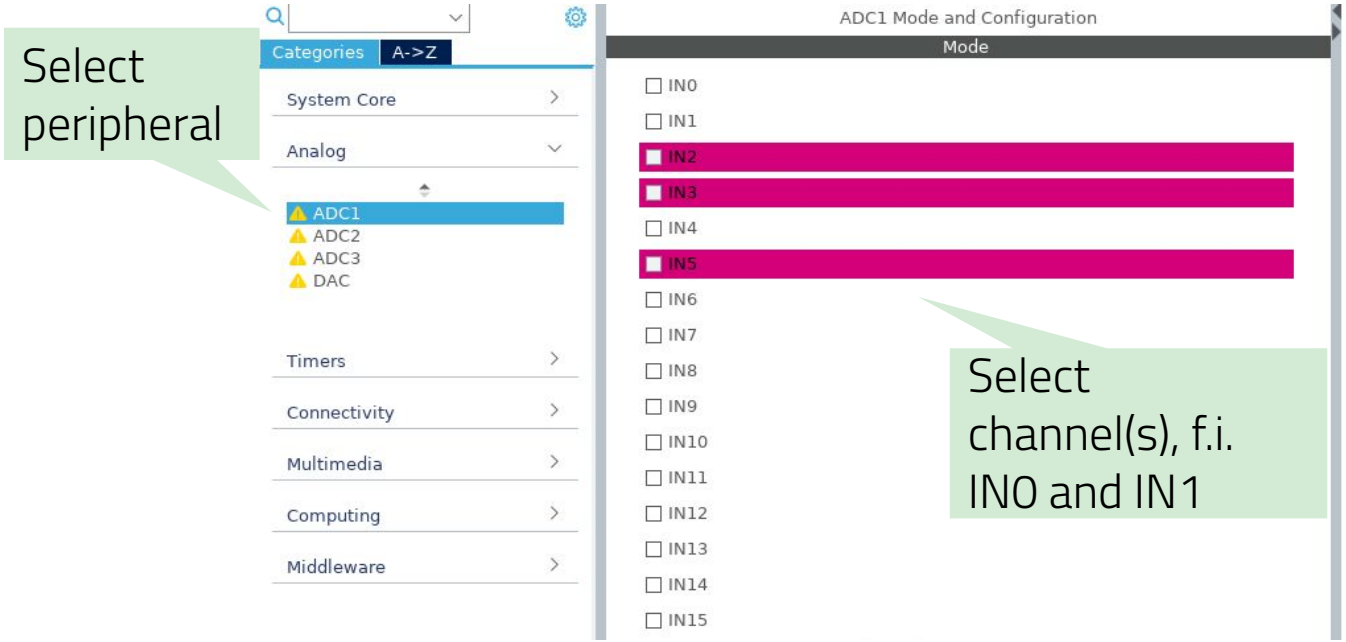
ADC Channel configuration

```
typedef struct {
uint32_t Channel; /* Channel ID: 0 - 15 */
uint32_t Rank;    /* Assigned rank: 1-16 */
uint32_t SamplingTime; /* Number of cycles for sampling*/
uint32_t Offset;  /* No use can be set to 0 */
} ADC_ChannelConfTypeDef;
```

How to use ADC in control loops

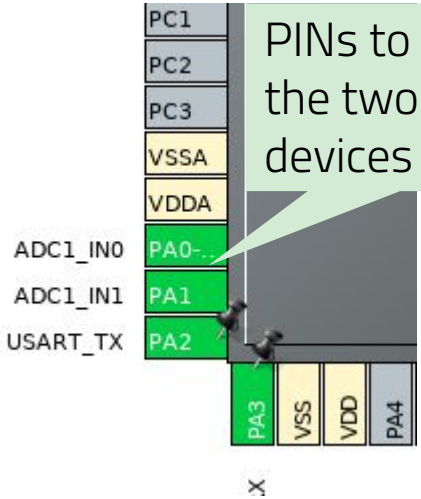
- In control loops when we want to read from a number of analog sensor continuously it makes sense to use single-channel or scan **continuous conversion** mode
- ADC readings can be clocked to specific timers to implement a precise sampling period
- As any other device, ADC can be accessed in polling, interrupt, or DMA mode
- We will explore access with DMA as an example

ADC configuration with CubeMX

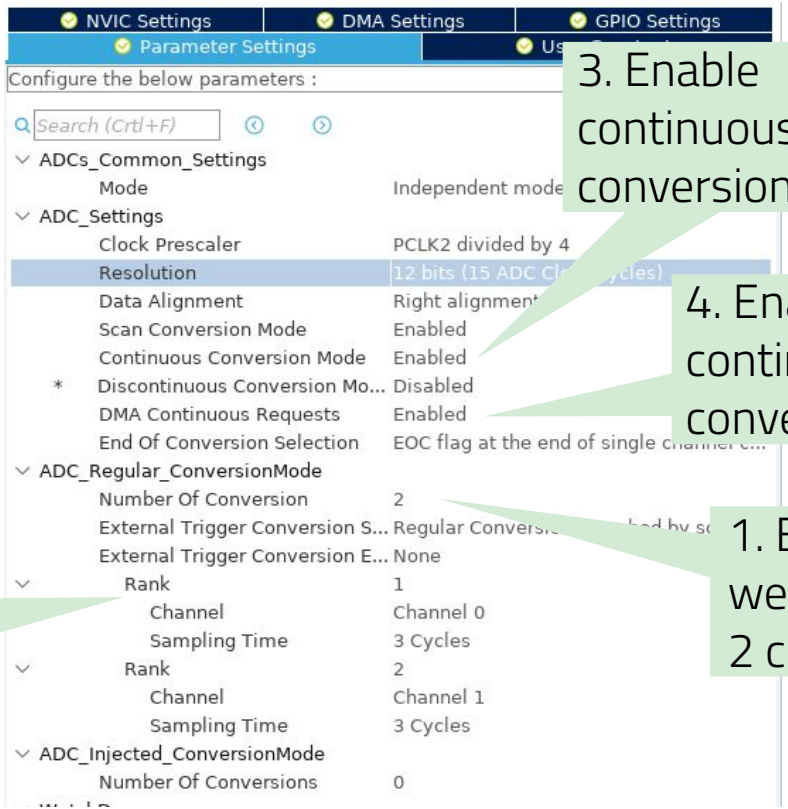


Example: configuring ADC1 to **scan continuously** channels `IN0` and `IN1` in **DMA** mode

ADC configuration with CubeMX



PINs to connect the two analog devices



3. Enable continuous conversion mode

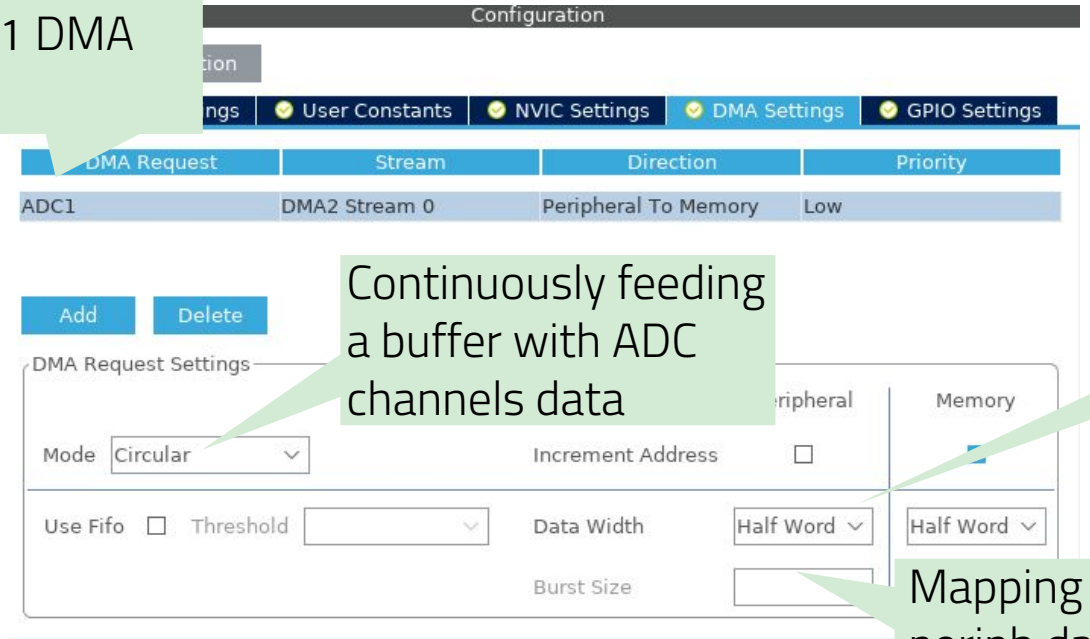
4. Enable continuous conversion mode

1. Enter **2** since we are capturing 2 channels

2. Set the rank (order) for the two channels

Configuring DMA

Add ADC1 DMA request

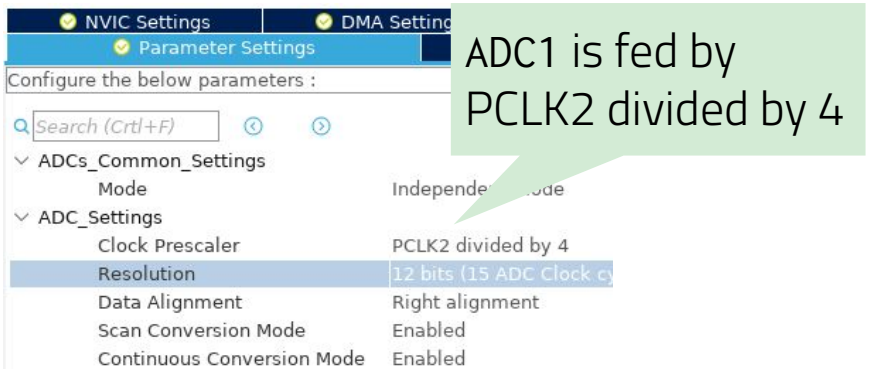


Continuously feeding a buffer with ADC channels data

Half-word since ADC produces 12-bit data

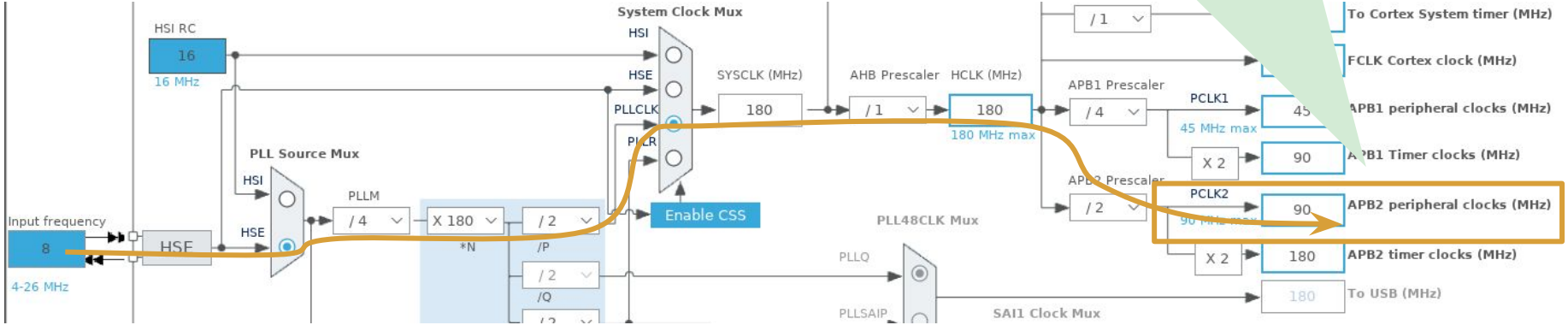
Mapping 16-bit of periph data to 16-bit in memory

How fast are we sampling?



ADC1 is fed by PCLK2 divided by 4

PCLK2 = 90MHz, ADC1 is fed at 90MHz/4 = 22.5MHz
We are sampling 2 channels each requiring 15 ADC clocks cycles + 3 for sampling, we have 22.5e6/(15+3)/2=625 kSPS



Start reading data in DMA circular mode

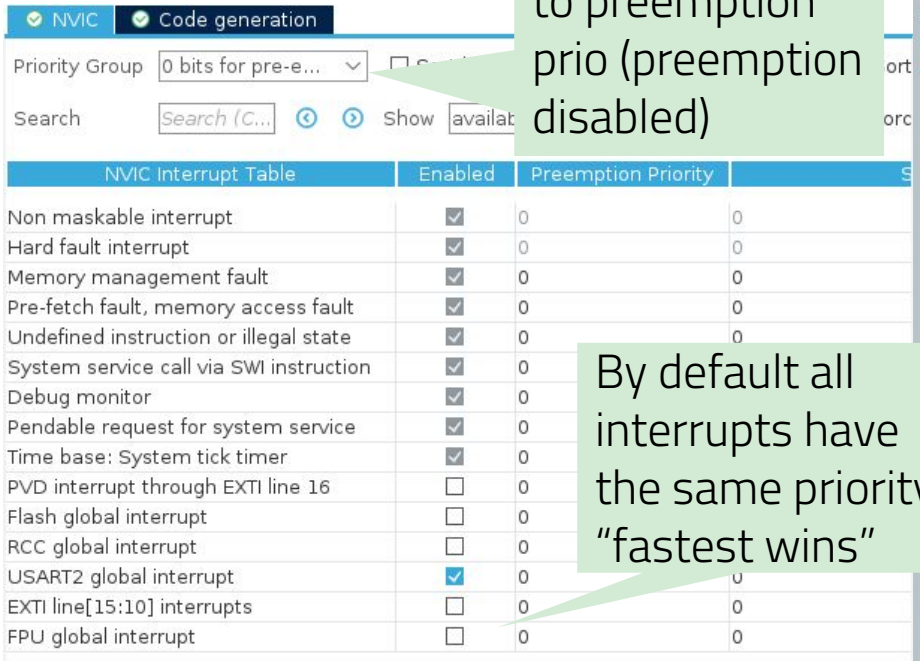
- It is as simple as starting the transfer, no other action is required
- `HAL_ADC_Start_DMA(ADC_HandleTypeDef* hadc, uint32_t* pData, uint32_t Length)`
- `pData`: destination buffer pointer, its type should match the configuration given in DMA data width (in the example it is half-word, 16-bit)
- `Length`: number of ADC readings for each DMA transfer. It should match the length of the buffer elements

Tip: too high data rates can be an issue

- DMA is typically used to fetch large amount of data in chunks
- DMA generates interrupts when a transfer is completed
- Using DMA at the fastest rate for short data transfer may not offer advantages over interrupt-based transmission
- Reading ADC at full speed might generate a too high interrupt rate that could “starve” (monopolize) execution of other important interrupts
- It is a good idea to lower the priority of DMA interrupts
- Beware that CubeMX defaults to 0 bits to preemption priority

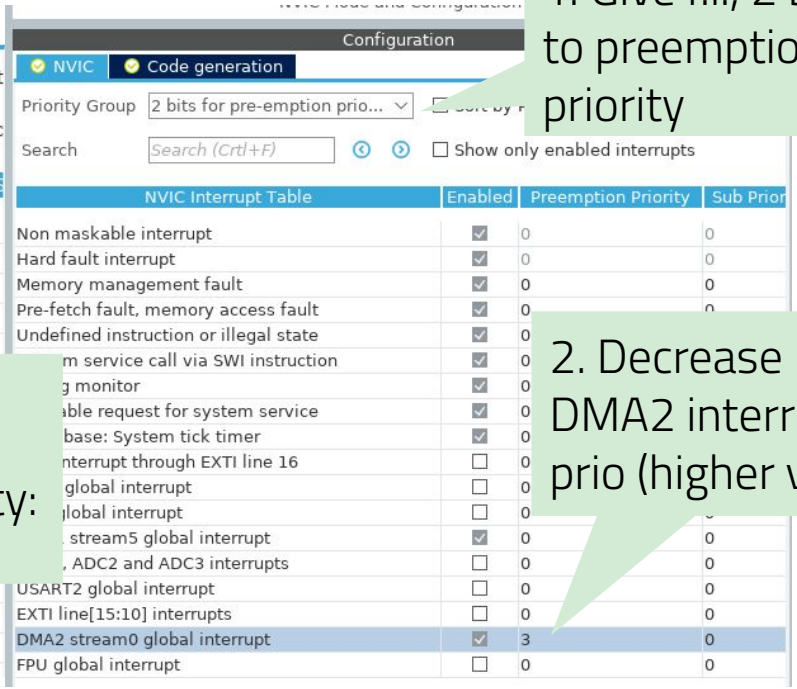
Modifying interrupt priority

By default 0 bits to preemption prio (preemption disabled)



By default all interrupts have the same priority: “fastest wins”

1. Give f.i., 2 bits to preemption priority



2. Decrease DMA2 interrupt prio (higher value)

Example: DMA circular mode

```
volatile uint16_t data[2]; /* 2 channels data is
                                    stored here */
void main() {
    …
 MX_ADC1_Init();
 HAL_ADC_Start_DMA(&hadc1, (uint32_t *)data, 2);
  while (1)   {
      /* Read sampled data each 100 ms */
      in0 = data[0];
      in1 = data[1];
      HAL_Delay(100); /* This might be starved */
}
```

ADC read in polling mode

- ADC, as any other device, can be accessed in busy-wait
- First, start ADC in polling mode with
  `HAL_ADC_Start(ADC_HandleTypeDef* hadc);`
- Then do a busy-wait loop waiting for new data to be produced using `HAL_ADC_PollForConverision(ADC_HandleTypeDef* hadc, uint32_t Timeout)`
- Then read the value:
  `uint32_t HAL_ADC_GetValue(ADC_HandleTypeDef* hadc)`

Example: polling mode

```
uint16_t adc1, adc2;
HAL_ADC_Start(&hadc1);
while(1) {
    HAL_ADC_PollForConversion(&hadc1, 100);
    adc1 = (uint16_t)HAL_ADC_GetValue(&hadc1);
    HAL_ADC_PollForConversion(&hadc1, 100);
    adc2 = (uint16_t)HAL_ADC_GetValue(&hadc1);
    HAL_Delay (100);
 }
```

Interrupt mode

- First, start ADC in interrupt mode with `HAL_ADC_Start_IT(ADC_HandleTypeDef* hadc);`
- Read data in the interrupt callback:

  `void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef* hadc)`
- To read the data always use `HAL_ADC_GetValue();`

Example: ADC in interrupt mode

```
volatile uint16_t rawValue;
void  main() {
...
HAL_ADC_Start_IT(&hadc1);
while (1) {};
}
/* Read data in the Conversion Complete callback */
void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef* hadc) {
    rawValue = (uint16_t)HAL_ADC_GetValue(&hadc1);
}
```